Introduction to Programming

Final Project
Personal Finance Manager



Department of Software Engineering-FIT-VNU-HCMUS

1

Basic features

Overview

Develop a Personal Finance Manager application in C++.

Detailed Requirements

1. **Transaction Management:**

The system allows users to record and manage daily income and expense transactions with a strict data structure distinguishing between the stored ID and the displayed name.

- Income Management:
  - Users enter information for an income transaction.
  - Required Data:
    - **Date:** The date the transaction is executed.
    - **Income Source:** The system stores the **Source ID** for logic processing but displays the **Source Name** on the user interface.
    - **Amount:** The value of the income.
    - **Wallet:** The system stores the **Wallet ID** but displays the **Wallet Name**.
    - **Description:** Detailed notes for the income.
- Expense Management:
  - Users enter information for expense transactions.
  - Required Data:
    - **Date:** The date the transaction is executed.
    - **Income Source:** the system stores the **Category ID** but displays the **Category Name**.
    - **Amount:** The value of the expense.
    - **Wallet:** The system stores the **Wallet ID** but displays the **Wallet Name**.
    - **Description:** Detailed notes for the expense.

2. Recurring Transactions Management

Support automation for entering frequently recurring income or expenses.

- Recurring Configuration**:** Allow pre-creation of recurring transactions on **Monthly** basis.
- Applicable Time:
  - Start Date**:** Required.
  - End Date**:** Optional (Can be left blank if the transaction recurs indefinitely).
- Mechanism**:** When the user opens the application, the system checks and adds the transaction for the current month if the conditions are met, ensuring that the transaction is not added again if it already exists for the current cycle.

3. Master Data Management

Provide functions to add, edit, and delete shared data categories to ensure consistency during data entry.

- Expense Categories
  - Manage a list of expense types (e.g., Food, Rent, Transportation).
  - Each category includes **Category Code (ID)** and **Category Name (Name)**.
- Income Sources
  - Manage a list of money sources (e.g., Salary, Bonus, Savings Interest).
  - Each source includes **Source Code (ID)** and **Source Name (Name)**.
- Wallet List (Wallets)
  - Manage money storage locations (e.g., Cash, Bank Account, Credit Card).
  - Each wallet includes **Wallet Code (ID)** and **Wallet Name (Name)**.

4. Statistics & Reporting

Provide an overview of financial status through aggregated figures.

- Time-based Statistics

- Calculate total **Income** and total **Expense** within a specific time range selected by the user (e.g., From Date A to Date B).
- Calculate net balance (Income - Expense) during that period.

- Time & Wallet-based Statistics
  - Filter Total **Income/Expense** by a specific time range.
  - Break down detailed figures by specific separate **Wallets** (e.g., View specifically how much the "Cash" wallet spent this month).
- Annual Income/Expense Overview - Year-based Statistics: Allow the user to select one or multiple years. Display **Total Income**, **Total Expense**, and **Net Balance** (Total Income - Total Expense) for the selected year(s).
- Annual Income Breakdown - Year-based Statistics: Allow the user to select one or multiple years. Display **Total Income** and the total amount aggregated by each **Income Source.**
- Annual Expense Breakdown - Year-based Statistics: Allow the user to select one or multiple years. Display **Total Expense** and the total amount aggregated by each **Expense Category.**

5. Data Persistence

- **File Storage:** All application data must be serialized and saved to a file using either **Binary** or **XML** format.
- **Session Persistence:** Ensure that data is preserved across sessions. The system must automatically load the saved data when the application is opened, ensuring that no information is lost when the user closes and restarts the application.

User Interface (UI) Requirements

1. Dashboard (Main Screen)

Upon launching the application, the interface must display a financial overview containing:

- **Total Balance:** The aggregate amount of money available across all wallets.
- **Wallet Balances:** A detailed list showing the current balance of each individual wallet (e.g., Cash, Bank Account).

- **Navigation Menu:** A menu system allowing users to access other functional modules (e.g., Add Transaction, Statistics, Settings).

2. Navigation Flow

Upon the completion of a specific function (e.g., successfully saving a transaction or finishing a configuration), the application must automatically navigate the user back to the **Dashboard** (Main Screen).

Technology Stack:

- **Programming Language: C++** for all application logic and implementation.
- **Object Modeling: Mandatory use of struct or class** to define data models for key entities (e.g., Transactions, Categories, Income Sources, and Wallets), ensuring data encapsulation and structure.
- **Data Structures:**
  - **Custom Dynamic Arrays:** Implement dynamic arrays manually using pointers and manual memory management (new, delete) to store lists of objects.
  - **Constraint:** Do not use Standard Template Library (STL) containers such as std::vector, std::list, or std::map.
- **String Handling:** The standard **std::string** class is permitted for handling text data (names, descriptions).
- **File Persistence:** Use the standard <fstream> library to implement **Binary** or **XML** file storage, ensuring data is saved and loaded to prevent loss between sessions.
- **User Interface: Console-based Interface (CLI)**. The application will render menus and data via text output and receive commands via standard keyboard input.

2

Submission

When submitting your project, it is essential to provide a well-organized zip file that includes all necessary components. Below are the detailed contents that should be included in the submission:

1. Source Code: create a structured folder hierarchy for your source code. A suggested structure might be:

```
PersonalFinanceManager/
├── src/                # Main source code files
│   ├── main.cpp        # Entry point of the application
│   ├── Income.cpp
│   ├── Expense.cpp
│   ├── ExpenseCategory.cpp
│   ├── IncomeSource.cpp
│   ├── ...             # Other necessary files
├── include/            # Header files
│   ├── Income.h
│   ├── Expense.h
│   ├── ExpenseCategory.h
│   ├── IncomeSource.h
│   ├── ...             # Other header files
├── assets/             # Asset files (images, sounds)
│   ├── stones/         # Chess stone images
│   ├── sounds/         # Sound effects and music files
└── README.md           # Brief description and instructions
```

2. Report Document

- **Title Page**: Title of the project, team members, and date of submission.
- **Project Overview**: A brief summary of the project, including objectives and goals.
- **Requirements Specification**: Detailed description of the application requirements, including all features implemented (as outlined in previous messages).
- **Design Document**: High-level architecture and design decisions. Include diagrams if necessary (e.g., UML diagrams for classes).
- **Implementation Details**: Brief description of how the code is organized, any specific libraries used, and major components of the codebase.
- **Testing**: Description of testing strategies and results. Include any known issues or limitations.
- **Future Improvements**: Suggestions for potential future work or enhancements to the application.
- **Demo videos:** Create video demonstrations covering all application functions. Upload these videos to **YouTube** and provide the links in your submission (e.g., included in the README.md file or a separate text document).

3. Work Division: create a document that outlines how work was divided among team members. This should include:

- **Team Members**: List all members involved in the project.
- **Responsibilities**: Describe the specific tasks each member was responsible for. For example:
  - Member A: Application Logic Implementation (Main.cpp, Income.cpp).
  - Member B: User Interface (UI.cpp, assets).
  - Member C: Testing and Documentation.
- **Collaboration Tools Used**
  - Mention any tools or platforms used for collaboration (e.g., Trello, Slack).
  - **GitHub (Mandatory):** The use of GitHub for version control is **required**.
    - Each team member must contribute via individual commits using their own account.
    - **Grading Policy:** Any member who fails to commit code to the repository will receive a score of **zero**.

4. Submission Instructions

1. **Create a Zip File**: Compress the entire project folder (e.g., StudentId1_StudentId2.zip).
2. **Double-Check Contents**: Ensure that all required files are included in the zip file.
3. **Upload**: Submit the zip file according to the specified submission guidelines provided by your instructor or project supervisor.